/Types, /Typing, and /Twats

From Foundations to Production Practice

Fahad Ahmed

Machine Learning Reply

March 2, 2026

/TypeTheory –review

What is a Type?

- Three perspectives on types:
  - Set-theoretic:
    - bool $= \{\text{True}, \text{False}\}$
    - int $= \mathbb{Z}$ (or bounded subset)
    - A value has type T if it belongs to set T
  - Proof-theoretic (Curry-Howard): type as a proposition; value of that type is a proof
  - Operational View: syntactic labels that let us prove the properties of a program before running it.
- A static guarantee $\Gamma \vdash e : T$

```
# Set-theoretic view in practice
type UserId = int
# But semantically: UserId      int
# Aim: can't pass arbitrary int where UserId expected
```

$$\texttt{UserId} \subset \texttt{int} \subset \mathbb{Z}$$

But semantically: $\texttt{UserId} \neq \texttt{int}$

Type Safety: Preventing Runtime Errors Statically (Op. Semantics)

Without types:

```
def process_user(user_id):
    return db.get_user(user_id).email.upper()

process_user("admin")  # Runtime: AttributeError (None.email)
process_user(12345)    # Runtime: AttributeError (no .email on int)
```

With types:

```
def process_user(user_id: int) -> str:
    user = db.get_user(user_id)
    if user is None:
        raise ValueError(f"User {user_id} not found")
    return user.email.upper()

process_user("admin")  # Static error BEFORE runtime
```

Operational Semantics Primer

- **Operational Semantics**: how expressions are evaluated *step by step*
- Everything is an expression/abstraction or a term ($t ::=$ from TAPL).
- Deconstruct $\mathbf{x} = \mathbf{2}$
- Evaluation: a reduction relation $\mathbf{e} \longrightarrow \mathbf{e}'$ materialized at runtime.

$$\frac{t_1 \rightarrow t_1'}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t_1' \text{ then } t_2 \text{ else } t_3}$$

- Values: expressions that cannot be further reduced.
- $t \longrightarrow^* t'$: t evaluates to t' in zero or more steps

What is a Type System?

- A type system is a static approximation of program behavior.
- Built atop Operational Semantics
- Fundamental Theorem: If $\vdash e : T$ and $e \longrightarrow *v$, then $\vdash v : T$
- Typing judgment: Under context Γ term t has type T

$$\Gamma \vdash t : T$$

$x : int, y : bool \vdash$ (if y then x else 0) : int

Type Safety

Two key theorems:

Progress

$$\emptyset \vdash t : T \Rightarrow \text{ either } t \text{ is a value or } t \to t'$$

Preservation

$$\Gamma \vdash t : T \ \wedge \ t \to t' \ \Rightarrow \ \Gamma \vdash t' : T$$

Soundness: Well-typed programs don't get stuck - Milner (1978)

Type Checking Machinery - Inference Rules (STLC)

- Typing Rules written as:

$$\frac{\text{premise}_1 \quad \text{premise}_2 \quad \ldots \quad \text{premise}_n}{\text{conclusion}} \quad \text{(RuleName)}$$

- An example rule for conditionals

$$\frac{\Gamma \vdash t_1 : \text{Bool} \quad \Gamma \vdash t_2 : T \quad \Gamma \vdash t_3 : T}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T}$$

- Therefore, type checkers are rule derivation engines.

Types for Function and Application

- Typing rule for functions:

$$\frac{\Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x : T_1 . t_2 : T_1 \to T_2}$$

- Typing rule for function applications:

$$\frac{\Gamma \vdash t_1 : T_1 \to T_2 \quad \Gamma \vdash t_2 : T_1}{\Gamma \vdash t_1 \ t_2 : T_2}$$

- Example term (STLC):

$$\lambda f : (\text{Bool} \to \text{Bool}).\ \lambda x : \text{Bool}.\ f(f(x))$$

- Formal notation: apply_twice : (int $\to$ int) $\to$ int $\to$ int ('type' of the function)

Python

```python
from collections.abc import Callable
def apply_twice(f: Callable[[int], int], x: int) -> int:
    return f(f(x))
```

Rust

```rust
fn apply_twice<F>(f: F, x: i32) -> i32
where F: Fn(i32) -> i32 {
    f(f(x))
```

/Type –landscape

Temporal Axis: Static vs Dynamic

Static Typing: Types checked *before* execution (compile time / analysis time) Examples: Java, C++, Rust, Go

Dynamic Typing: Types checked *during* execution (runtime) Examples: Python, JavaScript, Lisp
Key Takeaway: WHEN type errors are detected, not WHETHER

Strictness Axis: Strong vs Weak

- Strong Typing: Incompatible types cannot be implicitly coerced.
- Weak Typing: Implicit coercions may occur to continue evaluation.

JavaScript (weak):

```
"5" - 3       // -> 2   (string coerced to number)
"5" + 3       // -> "53" (number coerced to string)
"5" * 3       // -> 15
[] == false    // true  (coercion)
"" == 0        // true
```

Python (strong):

```
"5" + 3       # TypeError: can only concatenate str
"5" * 3       # "555" (well-defined, not coercion)
int("5") + 3 # 8 (explicit conversion required)
```

Type Systems Grid

	Static	**Dynamic**
Strong	Rust, Haskell Java, Go	Python Ruby
Weak	C, C++	JavaScript

Nominal vs Structural

- Nominal: Focuses on *naming* or explicit declaration
- Structural: Focuses on *shape*, *behavior*, or *feelings/vibes* (for GenZ)

```python
# nominal
class Cache:
    ...

class Redis(Cache):
    ...

# structural
class ObjectStore:
    def add(self): ...

class AzureBlobStorage:
    def add(self): ...
```

/Subtype – deepdive

The Subtype Relation

$S <: T$: if **S** is a subtype of **T**, **S** can be used *safely* in a context where a term of type **T** is ingested.

We define $<:$ as a binary relation on types with:

- Reflexivity: $T <: T$ (S-Refl)
- Transitivity: $\frac{S <: T \quad T <: U}{S <: U}$ (S-Trans)

Examples

- Guitar $<:$ Instrument
- int $<:$ float (not in Python)
- S3Bucket $<:$ DataStore

Subsumption rule: t has type S; S $<:$ T; t also has type T $\frac{\Gamma \vdash t : S \quad S <: T}{\Gamma \vdash t : T}$

Bottom Types

- $\bot <: T$ for all T (S-Bot)
- Subtype of every type
- Expressions that never return (infinite recursion, exceptions, crash)

```python
from typing import Never, NoReturn #https://typing.python.org/en/lates

def assert_never(arg: Never) -> Never:
    raise AssertionError("Expected code to be unreachable")

def stop() -> NoReturn: # function returning nothing returns 'None'
    raise RuntimeError('no way')
```

```rust
fn divide_by_zero() -> ! {  // Rust's Never type
    panic!("boom");
}
```

Top Types

- $T <: \top$ for all T (S-Top)
- Every type is a subtype of the top type
- Go: *interface*, *any*
- Python: *object*, *Any* (consitent-with, not true subtype relation)

```python
def func(x: object):
    return x
```

```go
func describe(v any) { // or interface{}
    fmt.Printf("%v (%T)\n", v, v)
}
```

Record Typing: Width

Record: objects or structs with named fields

Width Subtyping: A record with more fields is a subtype of one with fewer fields

$$\{l_1 : T_1, \ldots, l_n : T_n, l_{n+1} : T_{n+1}, \ldots, l_{n+k} : T_{n+k}\} <: \{l_1 : T_1, \ldots, l_n : T_n\} \text{ (S-RcdWidth)}$$

```
# If you have a function expecting {x:Nat, y:Nat}
def distance_from_origin(point: {x:Nat, y:Nat}) -> float:
    return sqrt(point.x**2 + point.y**2)

# You can pass {x:Nat, y:Nat, z:Nat}
point_3d = {x: 3, y: 4, z: 5}
distance_from_origin(point_3d)  # works      just ignores the z field
```

Key Takeaway: extra fields do not break anything!

Record Typing: Depth

Depth Typing: If each field type is a subtype, the whole record is a subtype

$$\frac{S_1 <: T_1 \quad \ldots \quad S_n <: T_n}{\{l_1 : S_1, \ldots, l_n : S_n\} <: \{l_1 : T_1, \ldots, l_n : T_n\}} \text{ (S-RcdDepth)}$$

Combining width and depth

$$\{name : str, age : int, ssn : string\} <: \{name : str, age : int\} <: \{name : string\}$$

Key Takeaway: more specific field types are safe!

Variance

Definition: Type-theoretic property of *type constructors* that determines how subtyping relationship propagate through parametrized types
Purpose: Substitutability under parametric abstraction

$$A <: B, \quad F[A] ? F[B], \quad \text{where} F[.] \text{ is a type constructor}$$

- Covariance: $F[A] <: F[B]$ e.g. Sequence[T] in Python, list[int] <: list[bool]
- Contravariance: $F[B] <: F[A]$

```
class DataStoreCollection:  def get(self) -> DataStore:  return DataStore()
class ObjectStoreCollection:  def get(self) -> S3Bucket:  return S3Bucket()
```

- Invariance: does NOT imply $F[A] <: F[B]$

Function Subtyping

Function types are: $T_1 \to T_2$

Subtyping rule: $\frac{S_1 <: T_1 \qquad T_2 <: S_2}{(T_1 \to T_2) <: (S_1 \to S_2)}$

- Parameters: **contravariant** $(S_1 <: T_1)$
- Returns: **covariant** $(T_2 <: S_2)$
- Example: Callable[[T], R]

Behavioral Subtyping

Barbara Liskov (1987):
If for each object o_1 of type S, there exists an object o_2 of type T such that for all programs P (

Simply: Wherever you use a 'T', you should be able to replace it with an 'S' and have the program **behave correctly** (not just type-check).

Key Semantics: This is more about semantics.

LSP Violation

```python
from dataclasses import dataclass
from typing import ClassVar, Any
from datetime import datetime

@dataclass(frozen=True)
class Entity:
    id: int

class Repository:
    def save(self, entity: Entity) -> None:
        """Idempotent method to save Entity"""
        raise NotImplementedError

class AuditedRepository(Repository):
    audited_log: ClassVar[list[str]] = []

    def __init__(self, db: Any) -> None:
        self.db = db

    def save(self, entity: Entity):
        self.db.save(entity)
        self.audited_log.append(f"Saved {entity.id} at {datetime.now()}")
        # NOT IDEMPOTENT - creates duplicate audit entries!
```

/GradualTyping

The Dynamic Type

- Every gradual typing system requires a wildcard.
- **Any**: A special type representing unknown/unchecked type

```python
from typing import Any
def f(x: Any) -> Any:
    return x + 1
f(2)
f('abc')
```

```rust
fn accept_anything(value: &dyn Any) {
    if let Some(num) = value.downcast_ref::<i32>() {
        println!("Got number: {}", num);
    }
}
}
```

Subtype-of vs Consistency-with

Consistent-with ($\sim$):

- If $T1 <: T2 \Rightarrow T1$ is consistent-with $T2$
- if T1 consistent-with T2, T1 can replace T2 *without conflict*
  int is consistent-with **float**
- Any is consistent-with every type
- Every type is consistent with Any

Typing Progression - example

```python
@app.route("/users/<user_id>")
def get_user(user_id):
    user = db.query("SELECT * FROM users WHERE id = ?", user_id)
    return jsonify(user)

# Step 1: Add Any
from typing import Any

@app.route("/users/<user_id>")
def get_user(user_id: str) -> Any:
    user = db.query("SELECT * FROM users WHERE id = ?", user_id)
    return jsonify(user)

# Step 2: Define models
from typing import TypedDict

class UserDict(TypedDict):
    id: int
    username: str
    email: str

@app.route("/users/<user_id>")
def get_user(user_id: str) -> UserDict:
    user = db.query("SELECT * FROM users WHERE id = ?", user_id)
    if user is None:
        abort(404)
    return user  # Type checker verifies this is UserDict
```

/Python –type-system

Nominal Subtyping: Explicit Opt-In

Python: via ABCs

```python
from abc import ABC, abstractmethod

class PaymentProcessor(ABC):
    @abstractmethod
    def process_payment(self, amount: Decimal) -> str:
        ...

    @abstractmethod
    def refund(self, transaction_id: str) -> None:
        ...

class StripeProcessor(PaymentProcessor):
    def process_payment(self, amount: Decimal) -> str:
        return "a string"

    def refund(self, transaction_id: str) -> None:
        return
```

Nominal Subtyping: Explicit Opt-In

Rust: via Traits

```rust
use rust_decimal::Decimal;

// ABC equivalent      trait defines the contract
trait PaymentProcessor {
    fn process_payment(&self, amount: Decimal) -> String;
    fn refund(&self, transaction_id: &str);
}

// Concrete implementation
struct StripeProcessor;

impl PaymentProcessor for StripeProcessor {
    fn process_payment(&self, amount: Decimal) -> String {
        "a string".to_string()
    }

    fn refund(&self, transaction_id: &str) {
        // no-op
    }
}
```

Virtual Subclassing with ABCs

When to use:

- Cannot modify third-party classes
- Want to fit existing code into ABC hierarchy
- Retroactive interface satisfaction

```
class PaymentProcessor(ABC):
@abstractmethod
def process_payment(self, amount: Decimal) -> str: ...

class ThirdPartyPayment:  # Doesn't inherit from PaymentProcessor
    def process_payment(self, amount: Decimal) -> str:
        return "external_txn"

# Register as virtual subclass
PaymentProcessor.register(ThirdPartyPayment)

isinstance(ThirdPartyPayment(), PaymentProcessor)  # True
```

Structural Subtyping - Python

Dynamic vs Static Protocols

```python
from typing import Protocol, Any
from msgspec import Struct

class CacheData(Struct):
    key: str
    value: Any

class CacheInterface(Protocol):
    async def aget(self, key: str) -> CacheData: ...
    async def aset(self, entry: Any) -> None: ...

class RedisCache:
    async def aget(self, key: str) -> CacheData:
        return CacheData(key=key, value='value')
    async def aset(self, entry: Any) -> None: return

async def func(cache: CacheInterface) -> None:
    await asyncio.gather(cache.aget('key1'), cache.aget('key2'))
```

Can we materialize this during runtime?

Structural Subtyping - Go

```go
type CacheData struct {
    Key string
    Value any
}

type CacheInterface interface {
    AGet(ctx context.Context, key string) (CacheData, error)
    ASet(ctx context.Context, value any) error
}

type RedisCache struct {}

func (r *RedisCache) AGet(ctx context.Context, key string) (CacheData, error) {
    return CacheData{Key: key, Value: "value"}, nil
}

func (r *RedisCache) ASet(ctx context.Context, entry any) error {
    return nil
}

func run(ctx context.Context, cache CacheInterface) error {
    ...
}
```

Collections - Python

```python
from collections.abc import Sequence
import collections
from typing import Any

class CustomSequence:
    def __init__(self, obj: Sequence[Any]):
        self._internal = obj

    def __len__(self) -> int:
        return len(self._internal)

    def __getitem__(self, idx: int):
        return self._internal[idx+1]

    def __contains__(self, value: Any) -> bool:
        return value in self._internal

seq = CustomSequence(('h', 'e', 'l', 'l', '\\0'))
print(isinstance(seq, collections.abc.Sized))
print(isinstance(seq, collections.abc.Container))
print(isinstance(seq, Sequence))
```

Collections Hierarchy in Python

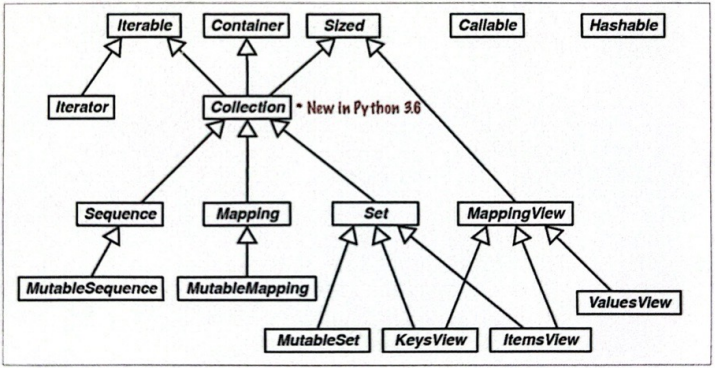


Figure 13-1. UML class diagram for ABCs in `collections.abc`.

Scanned with CamScanner

ABCs vs Protocols

Use ABCs when:

- You control the hierarchy
- Want explicit opt-in (nominal)
- Want to enforce abstract methods at instantiation
- Defining a taxonomy (Number, Sequence, etc.)

Use Protocols when:

- Describing shapes for duck typing
- Working with third-party code
- Want implicit satisfaction
- Maximum flexibility
- Retroactive interface satisfaction without registration
- No common initialization logic (super().__init__)
- Collapse multiple structures

Generic Types (Parametric Polymorphism) - Python

Purpose: Define type constructor that wrap and handle several types

```python
# PEP 484
from typing import TypeVar, Generic

T = TypeVar('T')

def identity(x: T) -> T:
    return x

class Box(Generic[T]): ...

# PEP 695
def identity[T](x: T) -> T:
    return x

class Box[T]: ...

class Handler[T, U]: ...

result = identity("hello")
result = identity[int](42)
```

Generic Types - Rust

```rust
fn identity<T>(x: T) {
    x
}

let text = identity("hello"); //inferred as string
let result = identity::<i32>(42); //explicit declaration

struct Box<T> {
    value: T
}

impl<T> Box<T> {
    ...
}
```

Bounded Generics

Example: $T <: \textit{Comparable}$

```python
# Pep 484
from typing import TypeVar
class Comparable(Protocol):
    def __lt__(self, other: Any) -> bool: ...

T = TypeVar('T', bound=Comparable)

def minimum(x: T, y: T) -> T:
    return x if x < y else y

# PEP 695
def minimum[T: Comparable](x: T, y: T) -> T:
    return x if x < y else y
```

Constrained Generics

Example: $T \in \{Comparable, str\}$

```python
# Pep 484
from typing import TypeVar

T = TypeVar('T', Comparable, str)

def minimum(x: T, y: T) -> T:
    return x if x < y else y

# PEP 695
def minimum[T: (Comparable, str)](x: T, y: T) -> T:
    return x if x < y else y
```

Can we use this during runtime? Let's switch to the terminal!

Subtyping with Generics - Python

```python
from abc import ABC, abstractmethod

class Repository[T](ABC):
@abstractmethod
async def get(self, id: int) -> T | None:
    """Fetch entity by ID."""
    ...

@abstractmethod
async def save(self, entity: T) -> None:
    """Save entity."""
    ...

@abstractmethod
async def delete(self, id: int) -> bool:
    """Delete entity. Returns True if deleted."""
    ...
```

Caveats:

- Tough to mock during tests
- Clients should be thoroughly onboarded.

Subtyping with Generics - Rust

```rust
pub trait Repository {
    type Entity;

    /// Fetch entity by ID
    fn get(&self, id: i32) -> Option<Self::Entity>;

    /// Save entity
    fn save(&self, entity: Self::Entity);

    /// Delete entity by ID. Returns true if deleted
    fn delete(&self, id: i32) -> bool;

    struct UserRepository;
}
impl Repository for UserRepository {
    type Entity = User;

    fn get(&self, id: i32) -> Option<Self::Entity> { ... }
    fn save(&self, entity: Self::Entity) { ... }
    fn delete(&self, id: i32) -> bool { ... }
}
```

/ProductionTwats

Questions?